

EC3404: GUI PROGRAMMING

Mini Project: The Modern Single Page Application (SPA)

CTF Knowledge Hub SPA

NAME : HERATH H.M.K.U.A.

REG NO : EG/2023/5634

DATE : 20/04/2026

Contents

1. Project Overview	3
2. Features Implemented	3
Core Features	3
Walkthrough System	3
Authentication.....	3
Bookmarks.....	4
Dark Mode.....	4
UI / UX Enhancements	4
3. Architecture & Component Design	4
Component Hierarchy	4
Composables.....	4
State Management (Pinia)	5
4. Technologies Used.....	5
5. API Integration.....	5
DummyJSON.....	5
Local Backend.....	5
6. Learning Outcomes	5
7. AI Usage.....	6
8. Screenshots	7
9. Conclusion	10

1. Project Overview

This project is a **Single Page Application (SPA)** developed using **Vue 3, TypeScript, and Tailwind CSS**.

The application simulates a **CTF (Capture The Flag) learning platform**, where users can:

- Browse cybersecurity challenges
- View detailed challenge descriptions
- Read and add walkthroughs
- Bookmark challenges
- Learn via curated resources

The system integrates both:

- **DummyJSON API** (for challenges & authentication)
- **Local backend (json-server)** (for walkthroughs)

2. Features Implemented

Core Features

- Fetch challenges from DummyJSON (/products)
- Display challenges in responsive cards
- Search challenges by title
- Filter challenges by category
- Challenge detail page (/challenge/:id)
- Dynamic routing using Vue Router

Walkthrough System

- Fetch walkthroughs from local backend (/posts)
- Display walkthroughs per challenge
- Add new walkthroughs dynamically
- View full walkthrough (/posts/:id)

Authentication

- Login using DummyJSON /auth/login
- Store user session in localStorage
- Manage login/logout using Pinia store

Bookmarks

- Add/remove bookmarked challenges
- Persist bookmarks using localStorage
- Global state using Pinia

Dark Mode

- Toggle between light and dark themes
- Implemented using Tailwind dark: classes
- Theme state managed via Pinia

UI / UX Enhancements

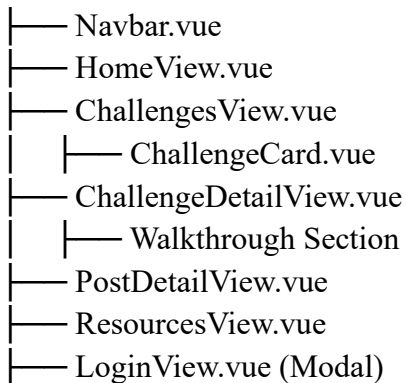
- Animated landing page
- Background image slider
- Smooth transitions and hover effects
- Responsive layout (mobile, tablet, desktop)
- Premium login modal overlay
- Resources page with embedded YouTube tutorials

3. Architecture & Component Design

The application follows a **component-based architecture** with separation of concerns.

Component Hierarchy

App.vue



Composables

Reusable logic is handled using composables:

- useChallenges.ts → Fetch & filter challenges
- useChallenge.ts → Single challenge data
- usePosts.ts → Manage walkthroughs

State Management (Pinia)

- auth.ts → Authentication
- theme.ts → Dark mode
- bookmarks.ts → Bookmark system

4. Technologies Used

- Vue 3 (Composition API)
- TypeScript (strict typing, no any)
- Tailwind CSS (utility-first styling)
- Pinia (state management)
- Vue Router (navigation)
- DummyJSON API
- json-server (local backend)

5. API Integration

DummyJSON

- /products → challenges
- /products/:id → challenge details
- /auth/login → authentication

Local Backend

- /posts → walkthroughs

6. Learning Outcomes

This project helped achieve:

- Strong understanding of **TypeScript interfaces**

- API handling using **async/await**
- Building scalable UI with **Vue components**
- State management with **Pinia**
- Responsive design using **Tailwind CSS**
- Effective use of **AI tools for development**

7. AI Usage

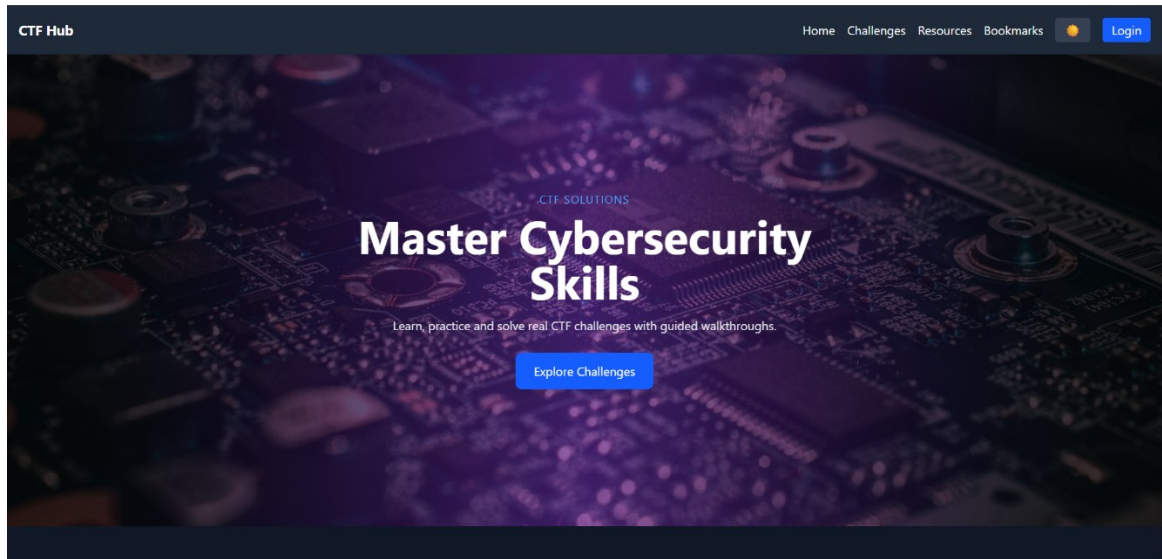
AI (ChatGPT) was used to:

- Debug TypeScript issues
- Improve UI design
- Generate Tailwind layouts
- Optimize architecture (composables & stores)

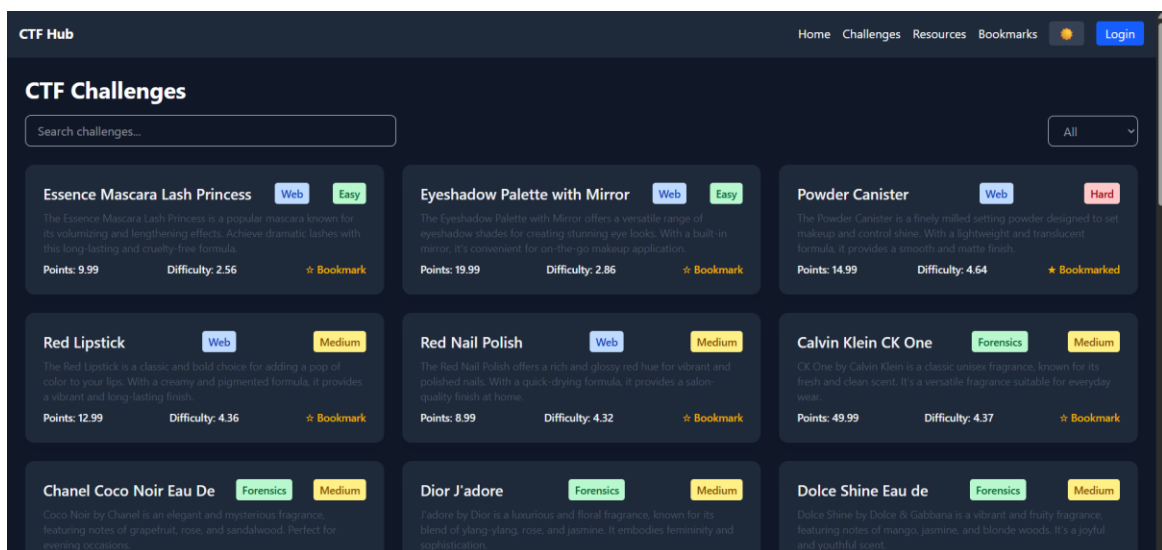
All prompts are documented in prompts.txt.

8. Screenshots

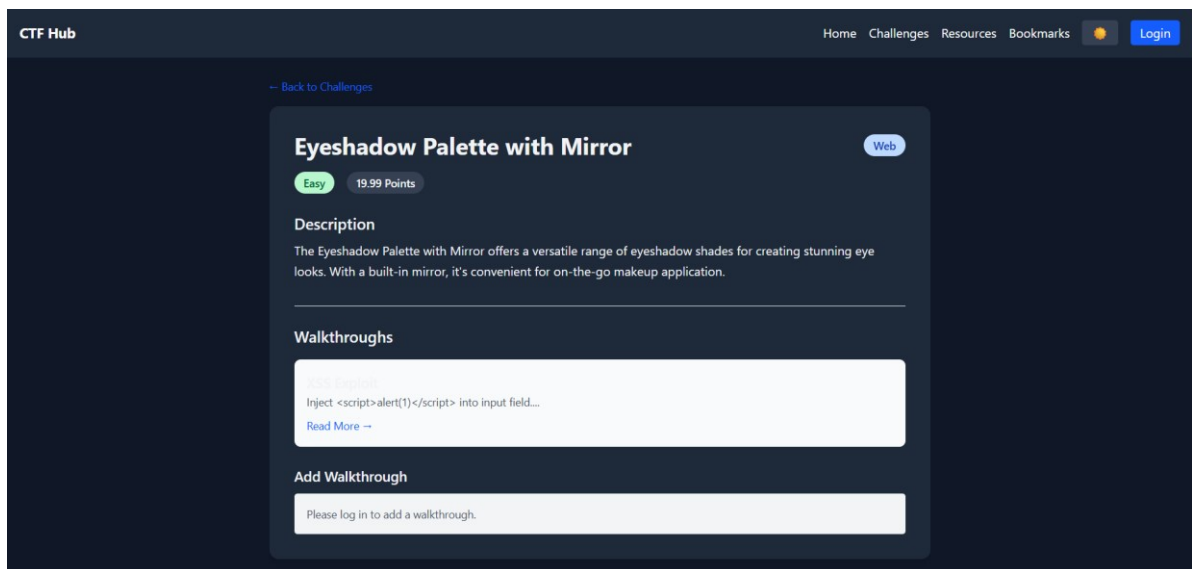
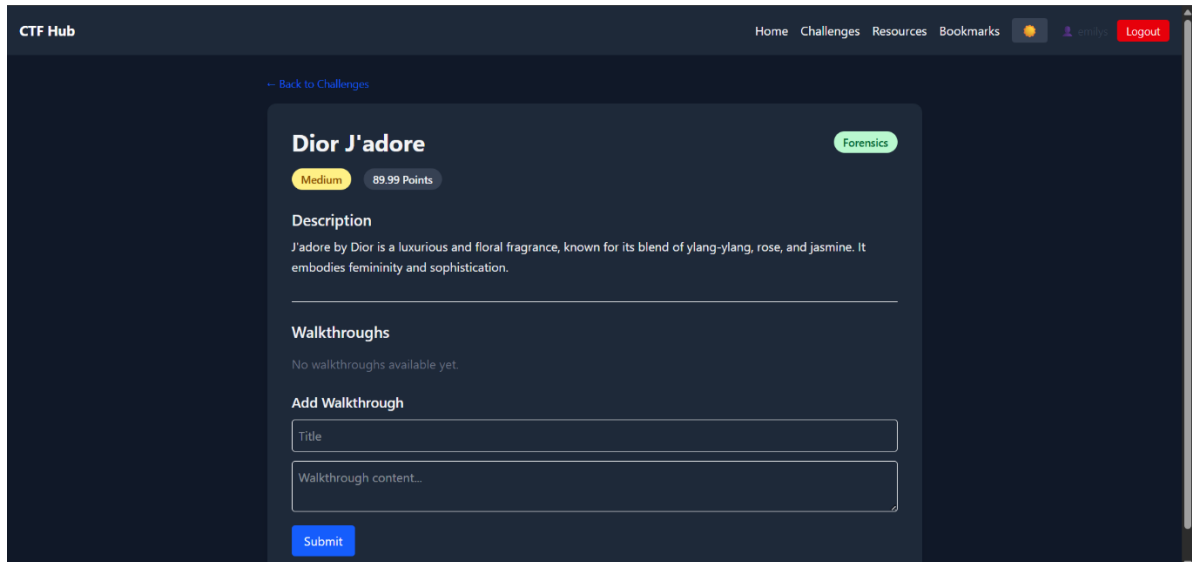
- Home Page



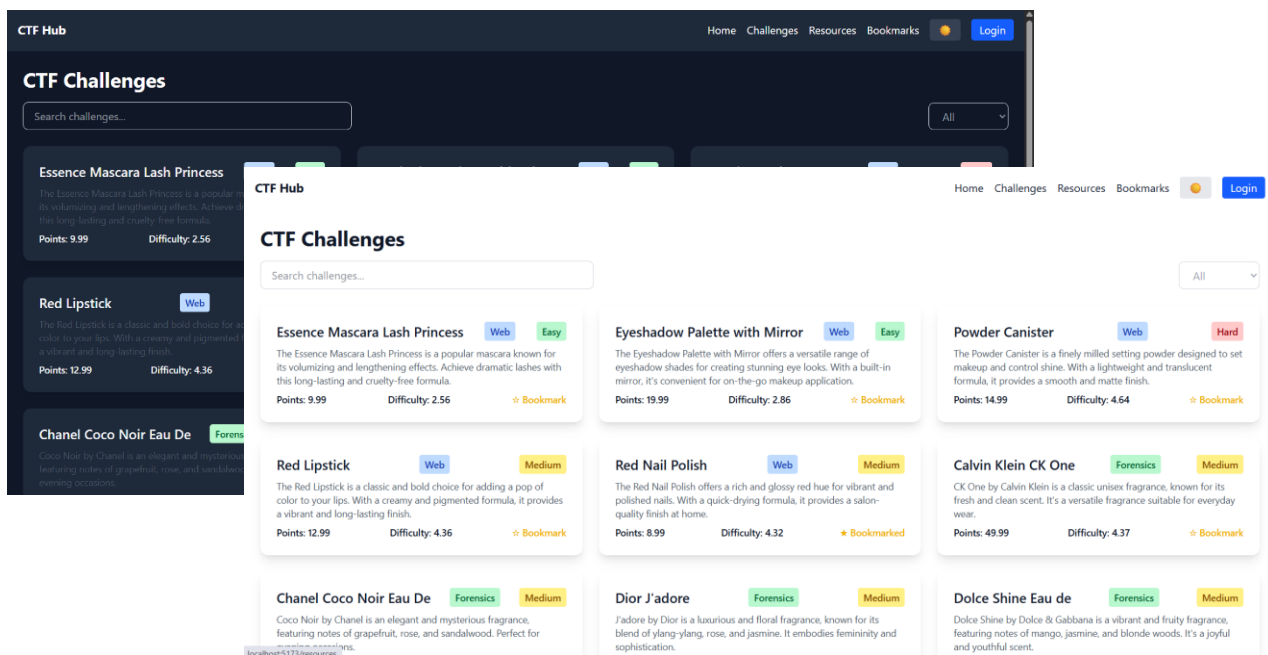
- Challenge List

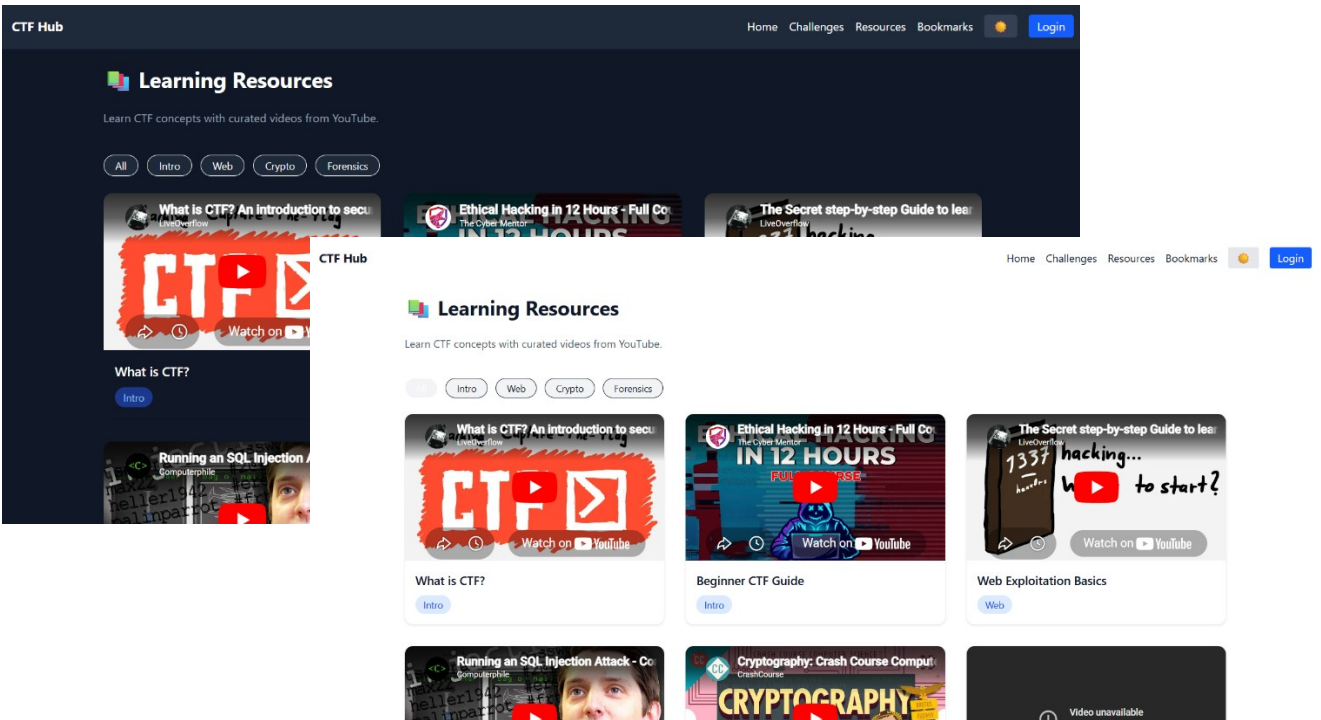
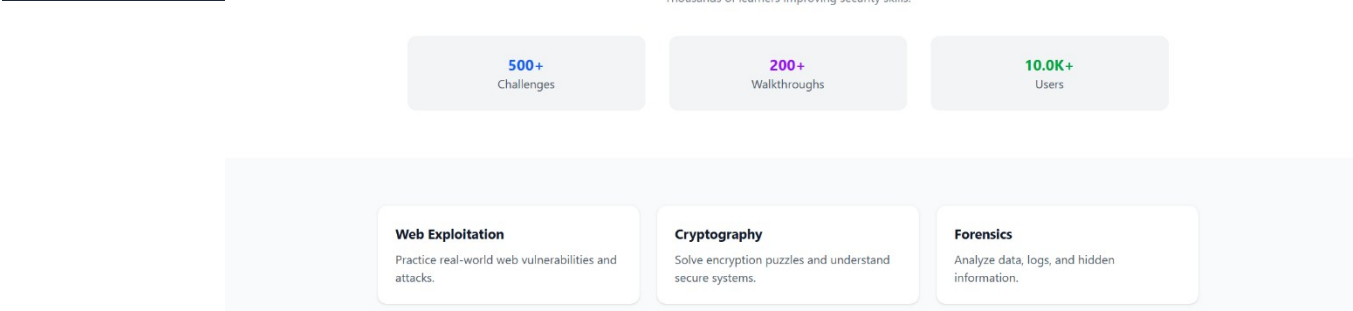
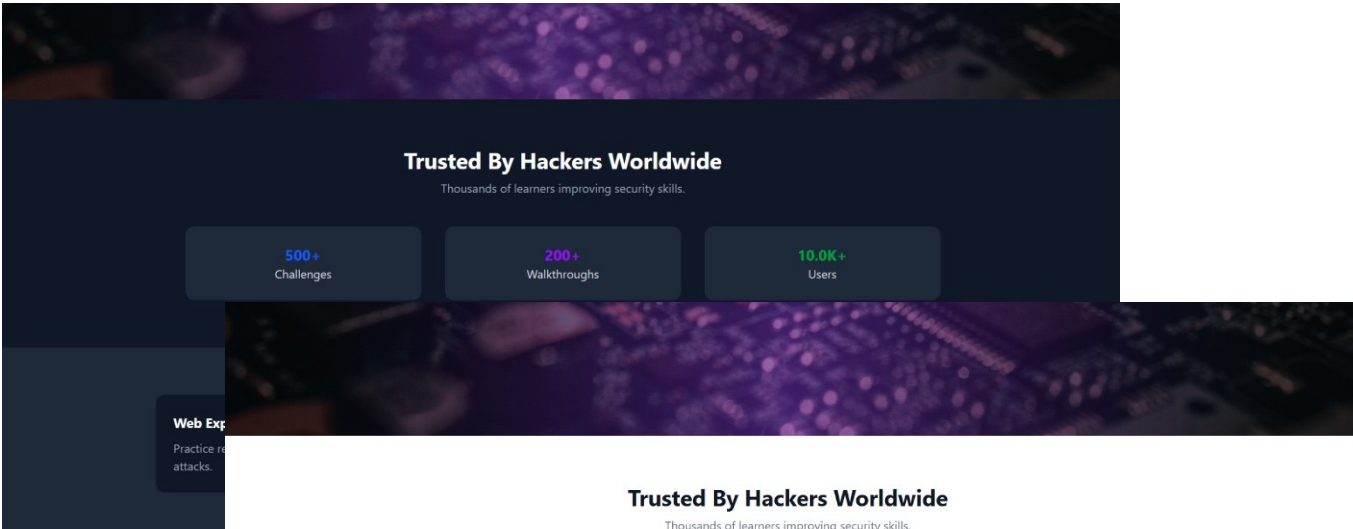


- Challenge Detail



- Dark Mode/Light Mode View





9. Conclusion

The application successfully demonstrates a **modern SPA architecture** with real-world features such as authentication, state management, and dynamic data handling.

It simulates a **production-level frontend system** with clean architecture and polished UI.